TEAM GUIDE // TWO PEOPLE, TWO CODING AGENTS

CO-OPERATIVE CODING

How any two-person team sets up their computers, agrees how to work, writes the plan and tasks, and wires up the repo and CI. Guess Closer! is the worked example.



TEAM
PARTNER A + B

AGENTS
ONE EACH

EXAMPLE
GUESS CLOSER!

REPO
github.com/Delta-43/cooperative-coding-workshop

HOW TO USE THIS GUIDE

Not a recipe
Every step says why. Swap in your own tools, stack and names.

Any agent
Claude Code, Codex, Cursor, Copilot, Gemini CLI. We use Claude Code for the demo.

One worked example
"Example" columns show what Guess Closer! chose.

CONTENTS

01 Set up each computer
02 The first conversation
03 PLAN.md and tasks
04 The repo
05 CI and branch rules
06 The working loop
07 Worked example
08 Safety, teardown, fixes

SECTION 01 · BOTH PARTNERS

SET UP EACH COMPUTER

Each partner works alone on their own machine, in their own clone, with their own agent. The setup is identical, so do it the same way and compare notes at the end.

01 **Accounts and access.** Both partners get write access to *one* repository, and nothing else. A fine-grained token can be limited to one repo (Contents, Issues and Pull requests, read and write, short expiry), but only for a repo that you or your organisation own, not one where you are a collaborator. So put the repo in a free organisation with both partners as members, or let the invited partner use an account that reaches nothing private. Test the sign-in with `gh auth status` before you start.

02 **Core tools.** Git, the language runtime your project needs (our example: Node.js 22 or newer) and the GitHub CLI. Check each with `--version`.

03 **A coding agent.** Any harness that reads files, runs commands and edits code under permission prompts. Pick one each; they do not have to match, as long as both read the same shared files.

04 **Instructions.** The agent reads a short, hand-written `AGENTS.md` at the repo root. If your harness uses its own file name, make that file point at `AGENTS.md` so there is one source of truth.

05 **Skills.** Add skills for jobs you repeat, such as contract review, code quality or design critique. Give each one a single job at a single moment, and have it report rather than rewrite.

06 **Connectors and MCP servers.** Add only what the task needs. Each one costs context and can carry instructions, so a smaller set is safer and cheaper.

07 **Guardrails.** Keep permission prompts on. Block the commands you never want run (deploys, force-push, reading `.env`) with permissions or a hook, because an instruction alone is only a request.

08 **Clone and run.** Clone, install, run the tests and start the project once, so you know the baseline works before the agent touches anything.

DONE WHEN (ON BOTH MACHINES)

- [ ] The project runs locally and its tests pass.
- [ ] Your agent can read `AGENTS.md` and tells you what it says.
- [ ] You can open and close a draft pull request from the terminal.
- [ ] Neither of you holds a credential the other one needs.

Never paste a token, password or `.env` file into an agent. If a task needs a secret, one person holds it and runs that step by hand.

SECTION 02 · TOGETHER, 20 TO 30 MINUTES

THE FIRST CONVERSATION

Agree these things in words, between people, before either agent writes a line. Write the answers into `PLAN.md` and `AGENTS.md` afterwards.

TOPIC	ASK EACH OTHER	EXAMPLE
Goal	What are we building, for whom, and what does "finished" look like in one sentence?	A live quiz game that a room of 30 can play.
Roles	Who is the coordinator who signs off? Who owns which part?	A: backend and sign-off. B: frontend and reviews A.
Seams	Where do our parts meet? What is the one interface we must agree first?	The WebSocket message format.
Ownership	Which folders does each person own? What is off-limits to the other?	`backend/` and `frontend/`; `contract/` shared.
Pace	How big is a task? How big is a pull request? How often do we sync?	A PR you can read in five minutes.
Review	Who reviews whom? What does "approved" mean? Who may merge?	The other person approves; agents only comment.
Agents	What may an agent do alone? What always needs a human? Which skills, when?	No deploys, no secrets; one fresh-context review per PR.
Decisions	Where do we record a decision so neither agent forgets it?	One line in `DECISIONS.md`, in the same PR.
Disagreement	What do we do when we disagree, or an agent surprises us?	Stop, talk, write the outcome down, then continue.

TALKING POINTS TO OPEN WITH

- [ ] "If the two agents meet only at the contract, what exactly goes in it?"
- [ ] "What would make you reject my pull request?"
- [ ] "Which part of this are we least sure about? Do that first, small."
- [ ] "Which of these decisions must the agents never make alone?"

SECTION 03 · TOGETHER

PLAN.MD AND TASKS

The plan is a short file that both agents read and only the humans change. It turns the conversation into phases, and each phase into tasks an agent can finish and a person can check.

PLAN.MD: WHAT GOES IN

```
# PLAN.md
Goal         one sentence, and what "finished" means
Team         Partner A, Partner B; who owns which folders
Design       the architecture in five lines or a small diagram
Contract     where the shared interface lives, and who may change it
Phases       1 contract · 2 build in parallel · 3 integrate · each with a sign-off
Out of scope  what we are deliberately not doing
```

A TASK (ONE ISSUE, ONE BRANCH, ONE AGENT)

Title — a verb and a result
Owner — one person and their agent
Done when — checkable in under a minute
Verify — the exact command
Do not touch — folders that are not yours

Why these five
"Done when" stops endless polishing. The verify command is the same for the agent, the human and CI. "Do not touch" keeps two agents out of each other's files.

EXAMPLE TASK AND PHASES

TASK	DONE WHEN	VERIFY	DO NOT TOUCH
B3 Scoring	Closest guess scores 3, then 2, then 1; equal error goes to the earlier answer	`cd backend && npm test`	`frontend/`
F2 Answer screen	Number field with the unit; send button stays above the phone keyboard	`cd frontend && npm run lint && npm run build`	`backend/`, `contract/`

PHASES AND SIGN-OFF

- [ ] **One sign-off issue per phase.** Only the coordinator closes it, after checking every "done when".
- [ ] **Phase 1, the contract.** Merge it first, on a skeleton, with both partners approving.
- [ ] **Phase 2, build in parallel.** Each partner works only in their own folders.
- [ ] **Phase 3, integrate.** Merge both sides and run one end-to-end check.
- [ ] **Small-change lane.** A typo or copy fix needs an issue and a PR, but no phase sign-off.

SECTION 04 · TOGETHER

THE REPO

Lay the repo out so that two agents rarely touch the same file. The folder tree is the ownership map.

```
repo/
├── AGENTS.md       how to run, test and name things (short, hand-written)
├── PLAN.md         goal, phases, who owns what (humans edit)
├── DECISIONS.md    one line per decision, with the date
├── contract/       the shared interface and an example of every message
├── backend/        Partner A
├── frontend/       Partner B
└── .github/workflows/ci.yml
```

FILE	KEEP IT	BECAUSE
`AGENTS.md`	Short and hand-written	A long generated file crowds out the task and does more harm than good.
`PLAN.md`	Owned by people	The plan must survive any single agent session.
`DECISIONS.md`	One line each	A new session has no memory; this file is the memory.
`contract/`	Examples that validate	CI can check the examples, so drift is caught early.

SET UP ON GITHUB, IN THIS ORDER

01 **Create the repo** and push the skeleton: folders, `AGENTS.md`, `PLAN.md`, `DECISIONS.md`, the contract and CI.

02 **Invite the partner** with write access to this repo only, on the day you start, not earlier.

03 **Turn on a branch rule for `main`:** a pull request is required, CI must pass, one approval is needed, and nobody pushes straight to `main`.

04 **Create milestones and issues** from `PLAN.md`: one milestone per phase, one "Phase N sign-off" issue each, and one issue per task with its done-when, verify and do-not-touch lines.

05 **Add labels** for owner and phase so each person sees only their own queue.

SECTION 05 · TOGETHER

CI AND BRANCH RULES

An instruction in a file is a request. A required check on the server is a guarantee. Put the rules that matter where the agent cannot skip them.

A MINIMAL, READ-ONLY CI (SHORTENED)

```
name: CI
on: { pull_request: , push: { branches: [main] } }
permissions: { contents: read }      # no secrets, no deploys
jobs:
  backend:    npm ci  →  npm test                # tests + contract examples
  frontend:   npm ci  →  npm run lint  →  npm run build
```

WHAT CI SHOULD CHECK

- [ ] Tests pass in each part of the repo.
- [ ] Every contract example validates against the schema.
- [ ] Lint and build pass.
- [ ] It runs the same commands as the issue's verify line.

WHAT IT SHOULD NOT DO

- [ ] Hold secrets or deploy.
- [ ] Be skippable by whoever wrote the code.
- [ ] Take more than a few minutes.

RULES FOR EVERY PULL REQUEST

BRANCH	One per issue, named after it: `b3-scoring`. One agent works on it.
SIZE	Small enough to read in five minutes. If it grows, split the issue.
CI	Must pass. The branch rule blocks the merge button until it does.
REVIEW	The other person approves. Agents may comment; a human approves.
EXPLAIN	The author explains the PR with the agent closed: what changed, and what breaks if it is wrong.
FRESH EYES	A read-only reviewer, in a clean context, checks the diff against the issue and reports. It does not edit.
DECISIONS	Any decision goes into `DECISIONS.md` as one line, in the same PR.

SECTION 06 · EACH PARTNER

THE WORKING LOOP

01 **Pick an issue** from your own queue and read its done-when and do-not-touch lines.

02 **Branch** named after it. Give the agent the issue, `AGENTS.md` and `PLAN.md`, and nothing more.

03 **Let the agent work** in your folders only. Run the verify command yourself.

04 **Run the skill** for this moment, such as contract review or design critique, and read what it reports.

05 **Open the PR** with the verify output. Wait for CI.

06 **The other person reviews.** You explain it with the agent closed. Merge only when CI is green and someone approved.

SECTION 07 · WORKED EXAMPLE

GUESS CLOSER!

A live quiz for a room. Players scan a QR code, pick a nickname and guess numbers on their phones; the closest guess wins. The projector shows the host screen.

```
Phones and laptops ──▶ GitHub Pages          static frontend (frontend/)
        │
        └── WebSocket ──▶ Cloudflare Worker ──▶ Durable Object, one per room
```

MESSAGE	FROM	WHAT IT DOES
`join · answer`	Player	Join with a nickname; send a number. A new answer replaces the old one and restarts its time.
`start · reveal · show · next · kick`	Host	Move the game along, or remove a player. `show` tells the server that the big screen now shows the answer.
`state · error`	Server	The whole game after every change. Players get the answer only after `show`, so no phone is ahead of the big screen.

Scoring. The closest guess scores 3, then 2, then 1. "Closest" is the smallest relative error: the distance divided by the answer. Equal error: the earlier answer ranks higher. Equal totals: the lower total answer time ranks higher.

Skills used in the demo. API contract review on the contract PR; a code-quality skill on backend PRs; design critique and animation review on the frontend. Each one reports, none rewrites.

SECTION 08

SAFETY AND TEARDOWN

- [ ] Give each agent the least access that works: one repo, a short-lived token, no real secrets in the folder.
- [ ] Everything an agent reads can carry instructions: issues, web pages, skills and MCP servers. Treat it as untrusted, and restate any request from outside before acting.
- [ ] Permission prompts stay on. No "skip permissions" mode outside a throwaway machine.
- [ ] Nothing reaches `main` without a PR, green CI and a human approval.
- [ ] You own what you ship: explain it, say AI helped, keep personal data out.

WHEN THE PROJECT ENDS

- [ ] Remove the partner's access and delete their token.
- [ ] Delete test deployments and rotate every token used.
- [ ] Read the repo's activity log for anything unexpected.
- [ ] Run `gh auth status` on each machine to confirm what is still signed in.

IF SOMETHING BREAKS

Two agents collide	Stop both. Split the work by folder or branch, then restart one.
CI red, agent says done	CI wins. Read the log, fix, push again.
A PR grew huge	Close it and split the issue.
The agent forgot a rule	It was only a request. Add a hook, a permission or a CI check.

APPENDIX

COMMANDS

```
# the flow (any stack)
gh issue list
git switch -c f2-answer-screen
gh pr create --fill
gh pr checks --watch
gh pr review --approve   # the other
partner
gh auth status

# example: backend
cd backend
npm ci
npm test
npm run dev

# example: frontend
cd frontend
npm ci
npm run dev
npm run lint
npm run build
```

PRE-FLIGHT CHECKLIST

- [ ] Both machines pass section 01.
- [ ] The first conversation is written into `PLAN.md`.
- [ ] The contract is merged before anyone builds.
- [ ] The branch rule and CI are on.
- [ ] Every task has done-when, verify and do-not-touch.